

MVC vs Razor Pages - A quick comparison

Published on Mar 22, 2018

I'm trying to learn ASP.NET but am really confused. All the tutorials seem to conflict with each other over whether I should be using MVC or Razor Pages?

If you're learning ASP.NET in this brave new .NET Core world and you want to build server-side web applications then it's a straight fight between two contenders.

Just to be clear, by sticking to "server-side" we're deliberately ignoring another option; building Web APIs in order to serve front-end frameworks like Angular.

In the blue corner we have MVC and in the red, the new kid on the block, Razor Pages.

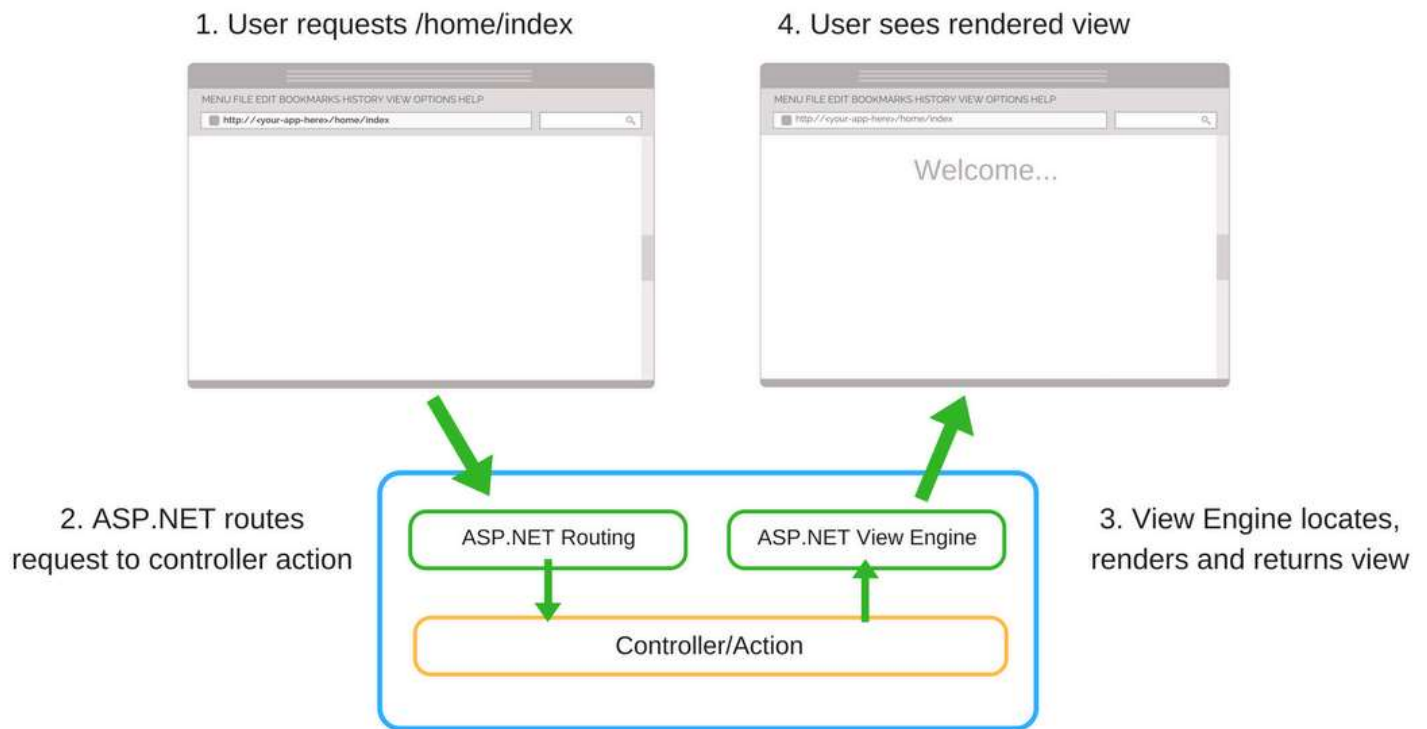
In truth they share a lot of the same underlying framework but there are some key differences.

Before you decide which one to adopt for your next project, it's worth taking a moment to directly compare them.

MVC

The old stager, been around since v1 all the way back in 2009.

Here's a rough overview of how MVC requests are handled by ASP.NET Core.



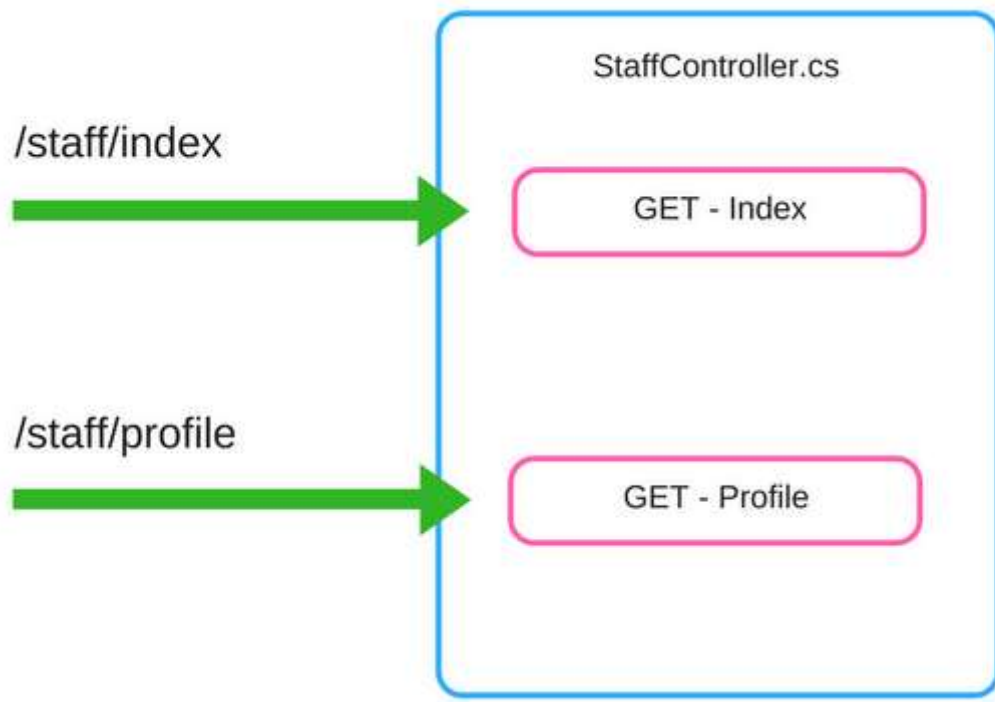
The routing engine is key to how ASP.NET Core decides to handle your requests.

It can be configured to route any request to any controller action.

The default routing configuration uses a combination of the controller and action names.

`/<controller name>/<action name>`

So a request to `/staff/index` would route to the action called `Index` on the `StaffController`.



```
public class StaffController : Controller
{
    [HttpGet]
    public IActionResult Index()
    {
        return View();
    }
}
```

Once the request reaches a controller action, the logic in the action is executed and a response is returned.

Where does my code go?

In MVC, your application code lives in the controller action.

```
public class StaffController : Controller
{
```

```
[HttpGet]
public IActionResult Index()
{
    // logic here...
    // calls to services etc.
    return View();
}
```

It's here that you're going to validate the request, execute any business logic, call your app's services etc, before eventually returning a response.

You can use attributes to indicate which kinds of requests your action will accept e.g. GET, POST, PUT (or any of the other HTTP verbs).

Where does my presentation code (markup) go?

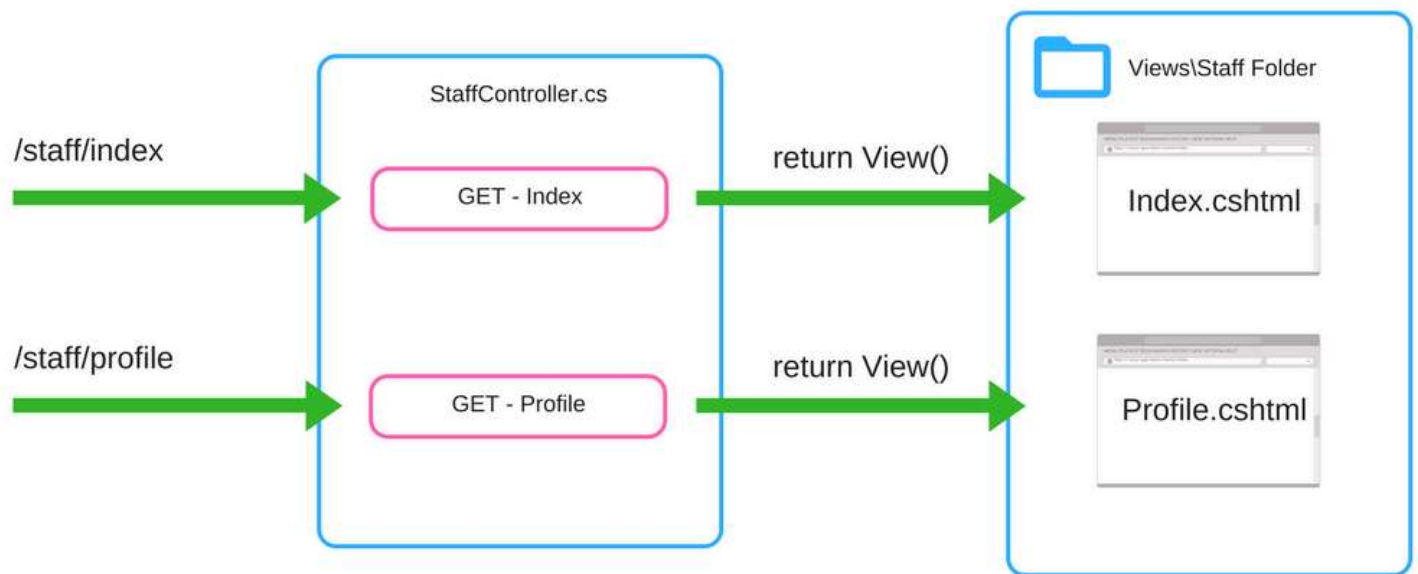
MVC actions don't have to return a View but they typically do.

It's in these views that you'll put all your HTML markup.

When a view is returned, MVC needs to locate said view and it achieves this using some default conventions.

Unless told otherwise, it will look in a `Views` folder for a folder with the same name as the controller, then a view with the same name as the action.

e.g. `Views\Staff\Index.cshtml`



The key thing to note here is that MVC requests are routed to the controller action and it's up to the action to determine whether a view is returned (and if so, which view).

MVC actions can easily override the default conventions and return a specific view (with a name which differs from that of the action).

```
public class StaffController : Controller
{
    [HttpGet]
    public IActionResult Index()
    {
        // logic here...
        return View("AnotherPage");
    }
}
```

This example will cause MVC to go looking for a view called `AnotherPage.cshtml` in the `Views\Staff` folder.

How do I display data in my views?

You can create a ViewModel. This is a class which lives in a .cs file and has properties for the data you wish to show on your view.

```
public class StaffProfile
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
}
```

Your controller actions can return this data alongside your views.

```
public class StaffController : Controller
{
    public IActionResult Index()
    {
        // would come from a database or something in real life
        var model = new StaffProfile { FirstName = "Jon", LastName = "Hilton" };
        return base.View(model);
    }
}
```

Your views can then render this data.

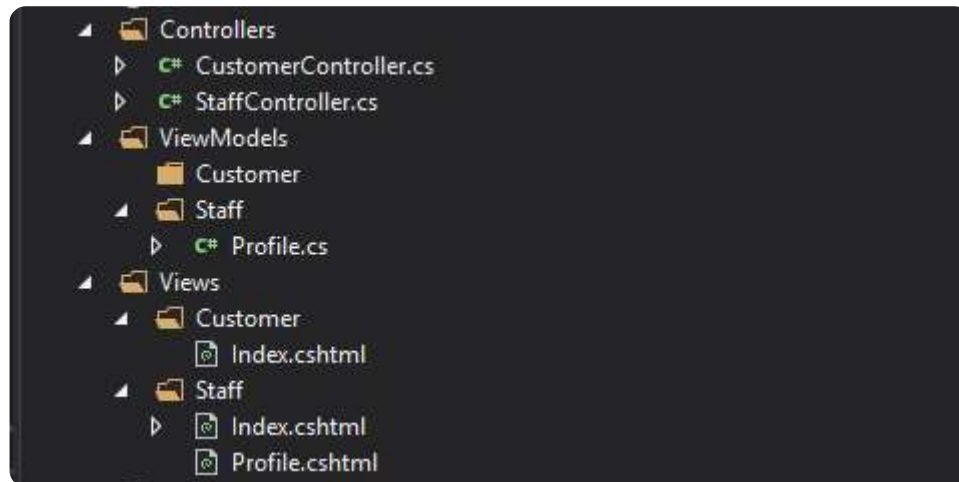
```
@model StaffProfile

<h1>Welcome</h1>

<p>Hey @Model.FirstName!</p>
```

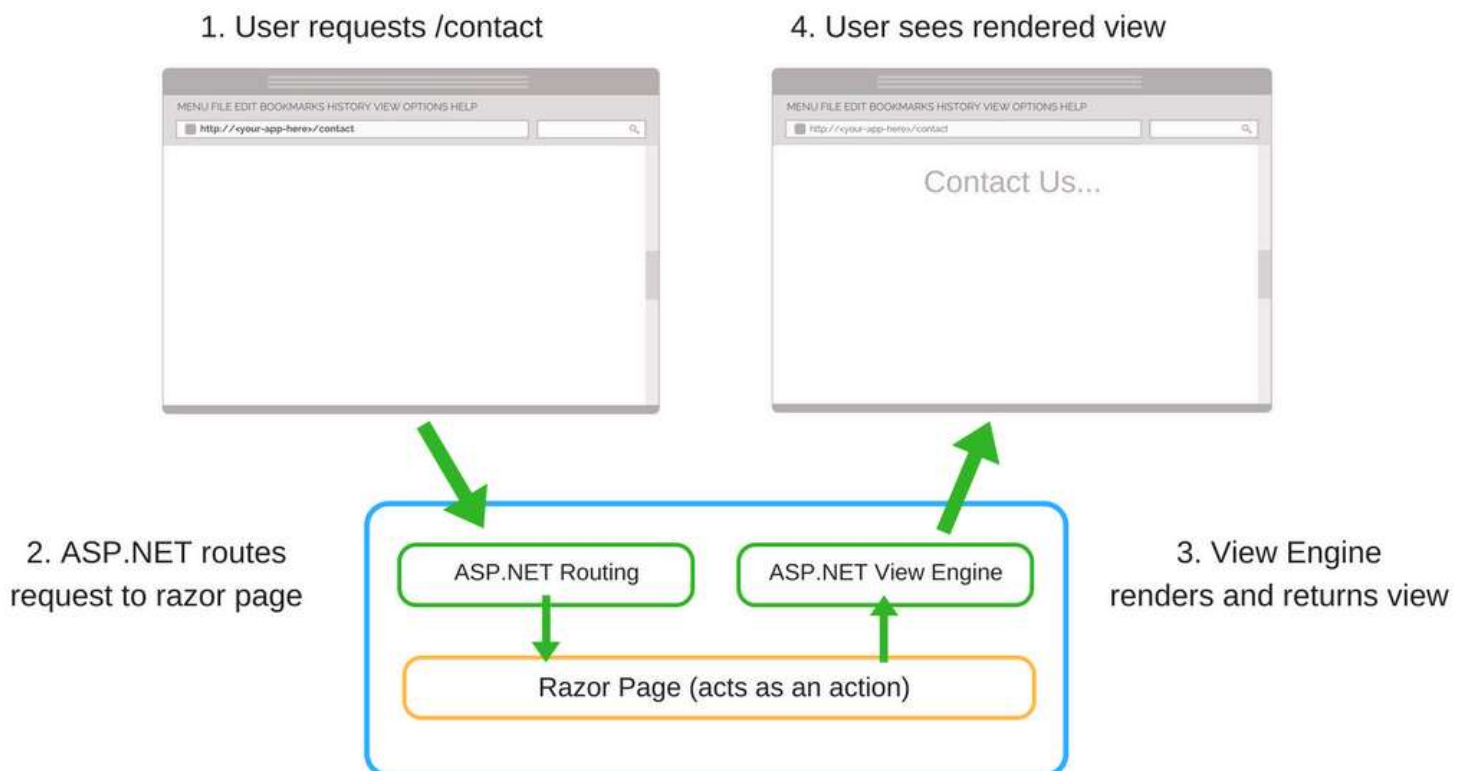
Folder structure

Typically, MVC apps have a standard folder-based convention with separate folders for controllers, views and view models.



Razor Pages

Razor Pages is brand new in ASP.NET Core.



With Razor Pages, when you make a request (e.g. `/contact`) the default ASP.NET

Jon Hilton - Making sense of ASP.NET



It simply looks for a page with the name used in the request (for a request to `/contact` that would be `Contact.cshtml`) and routes directly to it.

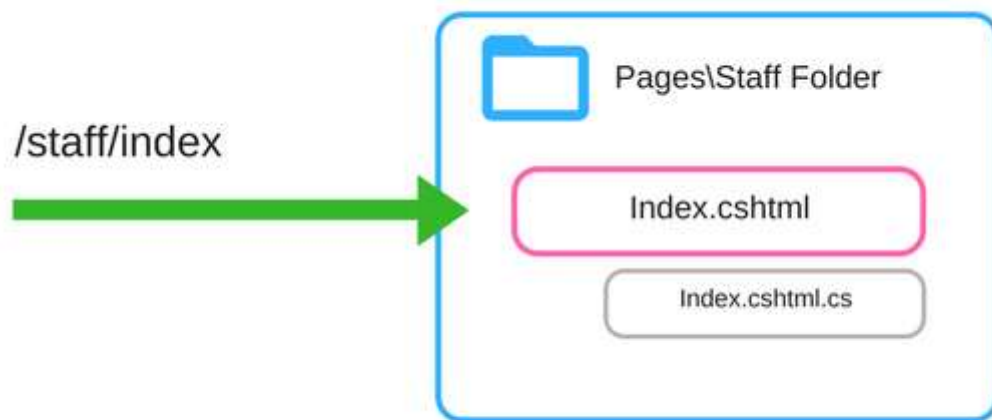
The Razor Page then acts as though it were a controller action.

For a `.cshtml` file to qualify as a Razor Page, it must live in the `Pages` folder (using default conventions) and include `@page` in its markup.

Where does my code go?

Every Razor Page can have a corresponding Page Model.

If we stick to the default conventions, ASP.NET will expect that model to have the same name as the corresponding Razor page but with a `.cs` appended.



With Razor Pages your application code lives in these models, specifically in methods for the different HTTP verbs (e.g. GET, POST).

```
public class ContactModel : PageModel
```



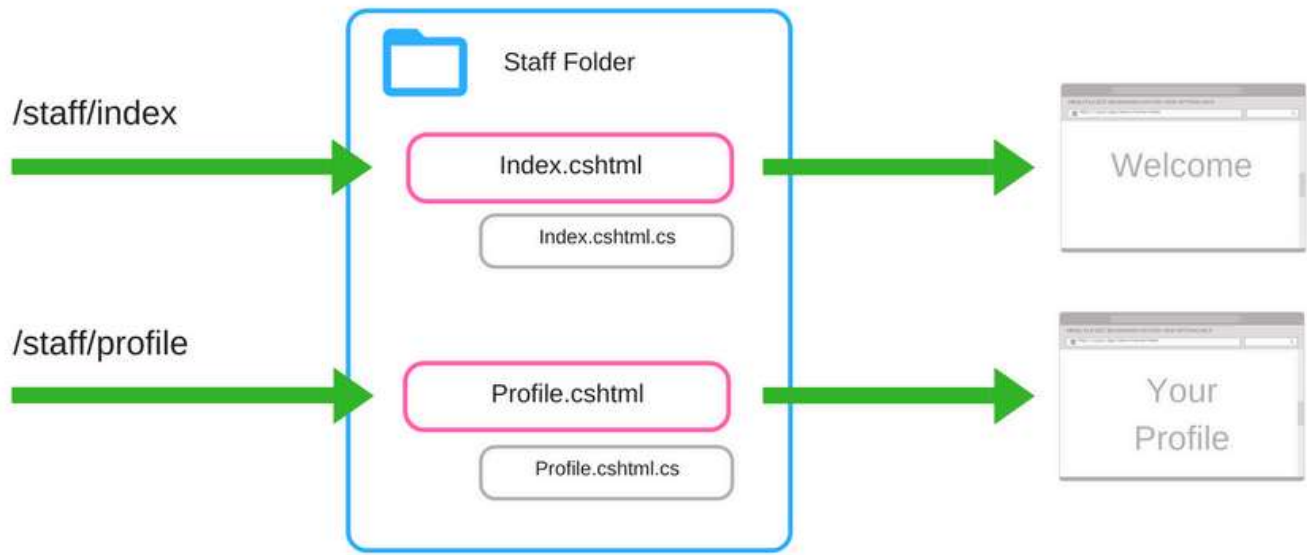
```
{  
    public void OnGet()  
    {  
        // logic here...  
        // calls to services etc.  
    }  
  
    public void OnPost()  
    {  
  
    }  
}
```

Where does my presentation code (markup) go?

Well this is simple.

Because the request was routed directly to the specific razor page that can handle it, there's no need to go off locating a view, the view is the one the request was routed to e.g. `Contact.cshtml`.

The default routing for Razor Pages is simple enough and respects subfolders, so `\Pages\Staff\Profile.cshtml` would be accessible via a request to `/staff/profile`.



Worth noting that you can omit `Index` as it is the default page. In this example both `/staff/` and `/staff/index` would take you to `Pages\Staff\Index.cshtml`.

How do I display data in my views?

Rather than have separate ViewModels you just need to add properties to your Page Model.

```
public class ProfileModel : PageModel
{
    public string FirstName { get; set; }
    public string LastName { get; set; }

    public void OnGet()
    {
        // would come from a database or something in real life
        FirstName = "Jon";
        LastName = "Hilton";
    }
}
```

```
}
```

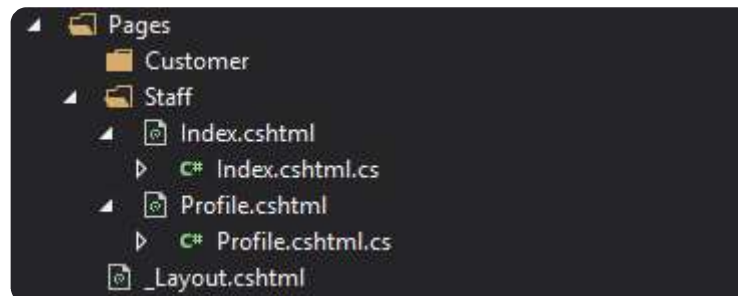
You can directly set those properties from your Page Model methods (e.g. OnGet).
Then it's simple to render this data in the Page itself.

```
@page  
@model ProfileModel  
  
<h1>Welcome</h1>  
  
<p>Hey @Model.FirstName!</p>
```

Folder structure

Everything lives in the Pages folder (by default).

Each Razor Page consists of the View template (.cshtml) and a corresponding .cs file which effectively acts as a controller action, specifically for that view.



In conclusion

So there you have it.

A quick comparison of MVC and Razor Pages.

It's early days for Razor Pages, they will almost certainly improve over the next few releases and it's not clear whether there is any kind of widescale adoption of in the industry.

What is certain is that MVC is here to stay. With so many existing apps written using it and the significant improvements which came with ASP.NET Core MVC, MVC isn't likely to disappear anytime soon.

Legacy .NET web apps causing you grief?

Build modern, reliable web applications, faster with .NET and Blazor.

Build better .NET web apps

Next Up

1. [Tackle more complex security policies for your ASP.NET Core app](#)
2. [Identify users and their permissions with JWTs and ASP.NET Core WebAPI](#)
3. [Pre-populate a textbox using ASP.NET Core MVC](#)

© 2025 Jon Hilton. All rights reserved.



[Contact](#) [Privacy Policy](#) [Cookie Policy](#)